

COMPLÉMENT AU RAPPORT TECHNIQUE

Kaay Dem !

Fonctionnalités ajoutées après la première version du rapport

Réalisé par : Aby Diaw — ISEP Diamniadio

Juillet 2026

Ce document complète le rapport technique initial (Rapport_Technique_KaayDem.pdf). Les sections 1 à 7 de ce dernier (contexte, architecture, modèle de données, diagrammes, sécurité) restent valables : elles ne sont pas reprises ici. Ce complément documente uniquement les fonctionnalités ajoutées ensuite, avec le même niveau de détail (quoi / pourquoi / mise en œuvre) que le corps du rapport.

1. Corrections et compléments aux exigences obligatoires

Deux manques ont été identifiés en comparant l'application à la version définitive du cahier des charges, et corrigés :

1.1. Signalement d'un utilisateur (EF-09)

Quoi : L'espace admin permettait déjà de consulter et traiter les signalements (GET/PATCH /admin/reports), mais aucune route ni aucun bouton ne permettait à un utilisateur d'en créer un.

Pourquoi : Le cahier des charges prévoit explicitement la « gestion des signalements d'abus » côté admin (EF-09) et le use-case « traiter les signalements » — cela suppose nécessairement qu'un signalement puisse être créé côté utilisateur, ce qui manquait.

Mise en œuvre : Nouvelle route POST /reservations/{id}/report, une ReportPolicy (seul un participant à une réservation confirmée ou terminée peut signaler l'autre partie), et un bouton « Signaler » sur la carte de réservation (passager) et sur chaque réservation reçue (conducteur).

1.2. Modifier / annuler un trajet publié (EF-03)

Quoi : Les routes PUT et DELETE /trips/{id} existaient déjà côté API (avec blocage si une réservation est confirmée, via TripPolicy), mais aucune interface ne les exposait : seule la clôture d'un trajet était possible depuis le tableau de bord.

Pourquoi : L'exigence EF-03 mentionne explicitement le CRUD complet des trajets, modification et annulation comprises.

Mise en œuvre : Ajout de deux boutons « Modifier » et « Annuler » sur chaque trajet publié dans le tableau de bord conducteur, visibles uniquement si aucune réservation n'est encore confirmée dessus (cohérent avec TripPolicy).

1.3. Cloche de notifications (EF-06)

Quoi : L'exigence EF-06 demande explicitement des « notifications visibles dans l'interface (cloche ou badge) », qui n'existaient pas du tout dans la première version.

Pourquoi : Plutôt qu'un flux d'évènements avec statut lu/non-lu (ce qui aurait demandé une table dédiée et une logique de marquage), on expose un compteur des éléments qui attendent réellement une action de l'utilisateur : demandes de réservation en attente côté conducteur, avis non encore laissés côté passager.

Mise en œuvre : Nouvelle route GET /me/notifications (NotificationController), et un composant NotificationBell dans l'en-tête, rafraîchi toutes les 30 secondes tant que l'utilisateur est connecté.

1.4. Modification du profil, y compris la photo (EF-01)

Quoi : La route PUT /me existait et acceptait déjà un champ photo, mais l'interface ne permettait de modifier que la lecture du profil, sans formulaire d'édition.

Pourquoi : L'exigence EF-01 mentionne explicitement un « profil modifiable (photo, téléphone, campus) ».

Mise en œuvre : Formulaire d'édition dans le tableau de bord, avec aperçu de la photo avant envoi.
Contrainte technique notable : un PUT multipart n'est pas fiable pour l'upload de fichiers en PHP ; le frontend envoie donc un POST avec un champ _method=PUT (spoofing de méthode standard de Laravel), qui route exactement vers le même contrôleur.

2. Nouvelles exigences bonus réalisées

Quatre exigences bonus supplémentaires ont été implémentées après la rédaction du rapport initial (qui n'en couvrait qu'une : la carte interactive).

2.1. Export CSV des trajets

Quoi : Un bouton dans l'espace admin permet de télécharger l'ensemble des trajets (y compris ceux annulés, via `withTrashed()`) au format CSV.

Pourquoi : Exigence bonus explicitement citée dans le cahier des charges.

Mise en œuvre : Contrôleur `AdminExportController` utilisant une `StreamedResponse` Laravel (réponse en flux plutôt qu'une chaîne construite entièrement en mémoire, pour rester léger même si le nombre de trajets grandit), avec un BOM UTF-8 pour un affichage correct des accents dans Excel. Le téléchargement est déclenché côté frontend via une requête axios en `responseType: 'blob'`, puisqu'un lien `<a href>` classique ne transmettrait pas le token d'authentification.

2.2. Notifications par e-mail via file d'attente Laravel

Quoi : Un e-mail est envoyé au passager quand le conducteur confirme ou refuse sa réservation.

Pourquoi : Exigence bonus explicitement citée (« notifications par e-mail via les files d'attente Laravel »), et objectif pédagogique implicite (traitement asynchrone).

Mise en œuvre : Deux `Mailable` (`ReservationConfirmeeMail`, `ReservationRefuseeMail`) implémentant `ShouldQueue` : l'envoi passe par la table `jobs` et un worker (`php artisan queue:work`) plutôt que de bloquer la requête HTTP. En développement, `MAIL_MAILER=log` écrit le contenu dans `storage/logs/laravel.log` sans nécessiter de vrai serveur SMTP.

2.3. Messagerie interne conducteur-passager

Quoi : Un panneau de discussion est disponible sur chaque réservation confirmée ou terminée, permettant au conducteur et au passager d'échanger des messages.

Pourquoi : Exigence bonus explicitement citée.

Mise en œuvre : Nouvelle table `messages` (`reservation_id`, `auteur_id`, `contenu`), une `MessagePolicy` (seuls les deux participants à la réservation, une fois confirmée, peuvent lire/écrire), et un composant `MessagePanel` côté frontend utilisant un polling simple (rafraîchissement toutes les 5 secondes tant que le panneau est ouvert) plutôt que des `WebSockets` — suffisant pour ce cas d'usage, sans infrastructure temps réel supplémentaire à déployer pour un projet académique.

2.4. Paiement simulé (portefeuille virtuel)

Quoi : Chaque utilisateur dispose d'un solde virtuel (10 000 FCFA à l'inscription) et d'un historique de transactions consultable depuis le tableau de bord.

Pourquoi : Exigence bonus explicitement citée (« paiement simulé des réservations, portefeuille virtuel »).

Mise en œuvre : Colonne `solde` sur `users`, table `transactions` (`montant`, `type` débit/crédit, `description`, réservation liée). Le montant total est débité du passager dès la demande de réservation (exactement comme les places sont retenues dès la demande), remboursé automatiquement si la demande est refusée ou annulée, et crédité au conducteur uniquement à la clôture du trajet — pas à la simple confirmation — pour refléter le fait qu'un paiement réel n'est débloqué qu'une fois le service rendu. Une nouvelle exception métier dédiée, `SoldeInsuffisantException` (409), suit exactement le même principe que `PlacesInsuffisantesException` déjà présente dans le projet.

3. Récapitulatif des exigences bonus

Exigence bonus	Statut
Carte interactive (Leaflet)	Réalisée (rapport initial)
Export CSV des trajets	Réalisée
Notifications par e-mail (file d'attente)	Réalisée
Messagerie interne conducteur-passager	Réalisée
Paieement simulé (portefeuille virtuel)	Réalisée
Mode hors-ligne (PWA)	Non réalisée
Déploiement en ligne	Non réalisé

4. Captures d'écran à ajouter

Les captures d'écran des sections déjà présentes dans le rapport initial (authentification, recherche, détail d'un trajet, tableau de bord, administration) restent valables dans leur principe, même si l'identité visuelle a été affinée depuis (nouvelle palette, typographie, icônes). Il est recommandé d'ajouter les captures suivantes, propres aux fonctionnalités de ce complément, avant la soutenance :

- Le bouton « Signaler » ouvert sur une réservation, avec le formulaire de motif
- Les boutons « Modifier » et « Annuler » sur un trajet publié, et le formulaire de modification ouvert
- La cloche de notifications avec un badge affichant un chiffre
- Le formulaire d'édition du profil avec l'aperçu de la photo
- Le bouton d'export CSV et le fichier téléchargé ouvert dans un tableur
- Le contenu d'un e-mail de confirmation, visible dans `storage/logs/laravel.log`
- Un échange de messages dans le panneau de messagerie
- La section « Portefeuille » du tableau de bord (solde + historique des transactions)

5. Conclusion (complément)

Avec ces ajouts, le projet Kaay Dem ! couvre désormais l'intégralité des exigences obligatoires (EF-01 à EF-09) ainsi que cinq des six exigences bonus proposées par le cahier des charges. Les deux manques identifiés a posteriori (signalement côté utilisateur, cloche de notifications) ont été corrigés, et quatre exigences bonus supplémentaires ont été implémentées : export CSV, e-mails en file d'attente, messagerie interne et paiement simulé. Le projet est structuré en dépôt Git avec un historique de branches et de commits conventionnels (voir CONTRIBUTING.md), prêt pour une revue par les pairs via des Pull Requests GitHub.